Audit Analytics & Internal Control Monitoring System

Deloitte-oriented flagship Data Analyst portfolio project — complete concept, features, architecture, implementation plan, and interview preparation

1. Project Objective

Build an end-to-end audit analytics application that ingests synthetic financial data, validates and transforms it, stores it in PostgreSQL, applies SQL-based audit tests and statistical/ML anomaly detection, assigns explainable risk scores, and presents exceptions through dashboards and an investigation workflow.

Core question: Which transactions deserve attention, why were they flagged, how risky are they, and what financial exposure might they represent?

2. Recommended Technology Stack

Area	Technology
Backend	Python, Django
Database	PostgreSQL
Analytics / ETL	Pandas, NumPy
SQL	PostgreSQL SQL, joins, CTEs, window functions, aggregations
Machine Learning	Scikit-learn — Isolation Forest
APIs	Django REST Framework (only if implemented)
Frontend	HTML, CSS, JavaScript, Django Templates
Visualization	Power BI + Django dashboard
Testing	Django/Python unit tests
Version Control	Git / GitHub

Important stack decision

Do not add FastAPI. Django is sufficient for authentication, RBAC, database models, APIs, validation, dashboards, and the web application. Add Power BI because it materially strengthens the Data Analyst positioning. Add Django REST Framework only if you actually implement and understand it.

3. Business Scenario

Use a fictional company such as **ABC Manufacturing Pvt. Ltd.** and generate synthetic data for demonstration. Do not represent the data as real company information.

- Departments: Finance, Procurement, IT, HR, Operations
- Regions and cost centers
- Employees and managers
- Vendors and vendor master data
- Purchase orders
- Invoices
- Payments
- Employee expenses
- Audit exceptions and review records

4. End-to-End Architecture

CSV / JSON / XLSX → Django upload → file/schema validation → Pandas ETL → PostgreSQL → SQL analytics → audit rules → statistical analysis → Isolation Forest → risk scoring → audit exceptions → investigation workflow → Django dashboard / Power BI → reports.

5. Database Design

Table	Important fields
employees	employee_id, name, department_id, designation, region, manager_id, joining_date, status
departments	department_id, department_name, cost_center, manager_id
vendors	vendor_id, vendor_name, vendor_type, region, bank_account, registration_date, risk_category, status
purchase_orders	po_id, vendor_id, employee_id, department_id, po_date, amount, approval_status, approved_by
invoices	invoice_id, vendor_id, po_id, invoice_date, due_date, amount, tax_amount, payment_status
payments	payment_id, invoice_id, vendor_id, payment_date, amount, payment_method, bank_account, processed_by
employee_expenses	expense_id, employee_id, department_id, expense_date, category, amount, description, approval_status, approved_by
audit_exceptions	exception_id, transaction_id, exception_type, severity, risk_score, description, detected_date, status, assigned_to, review
audit_logs	user, action, timestamp, entity, old_value, new_value

6. Audit Rules / Features

- Duplicate invoice detection — same/similar vendor, date, amount and reference characteristics.
- Duplicate payment detection — repeated payment for the same invoice/vendor/amount.
- Weekend transaction detection — flags transactions occurring on Saturday/Sunday for review.
- After-hours transaction detection — flags transactions outside defined operating hours.
- Round-amount detection — identifies large, perfectly rounded transaction amounts for review.
- Approval-threshold violation — compares transaction amount with the required approval level.
- Split-purchase detection — identifies several related purchases that collectively exceed an approval threshold.
- Unusual vendor activity — compares current vendor behavior with historical patterns.
- Employee expense anomaly — identifies expenses unusually large relative to an employee's history.
- Duplicate employee expense — detects repeated employee/date/amount/description patterns.
- Employee-vendor relationship check — flags potential relationships requiring investigation, without treating them as proof of misconduct.

7. Risk Scoring

Use a transparent, explainable score rather than relying entirely on ML. Combine rule severity, transaction value, frequency, historical deviation, and control impact. Normalize the result to 0–100.

- 0–30: Low
- 31–60: Medium
- 61–80: High
- 81–100: Critical

The exact weights must be documented in the repository and should be based on the rules you actually implement. An exception is a review signal, not proof of fraud.

8. Machine Learning Layer

Isolation Forest is appropriate as an additional unsupervised anomaly-detection layer when labeled fraud outcomes are unavailable. Possible features include transaction amount, transaction frequency, vendor transaction count, employee

transaction count, days since previous transaction, average vendor amount, and amount deviation.

Recommended flow: rule-based score + statistical signals + ML anomaly score → final explainable risk score. Keep the rule-based layer prominent because audit users need understandable reasons for flags.

9. SQL Analytics

Build approximately 20–30 meaningful queries and document them in GitHub. Cover joins, GROUP BY/HAVING, subqueries, CTEs, window functions, date analysis, duplicate detection, ranking, aggregation, and exception rates.

Example duplicate-invoice query:

```
SELECT vendor_id, invoice_date, amount, COUNT(*) AS invoice_count FROM invoices GROUP BY
vendor_id, invoice_date, amount HAVING COUNT(*) > 1;
```

Example vendor-payment analysis:

```
SELECT vendor_id, SUM(amount) AS total_payment, COUNT(*) AS payment_count FROM payments GROUP
BY vendor_id ORDER BY total_payment DESC;
```

10. ETL and Data Quality

- Accept CSV, JSON and XLSX uploads through Django.
- Validate file type, file size, required columns and data types.
- Clean duplicates, missing values, dates, currency values, IDs and categories.
- Produce a data-quality report before loading.
- Load clean records into PostgreSQL.
- Track valid rows, rejected rows, duplicates, missing values and invalid records using actual measured results.

11. Django Application Modules

- accounts — authentication and role management
- transactions — financial records and transaction views
- analytics — ETL, statistics and SQL analytics
- audit_engine — audit rules, risk scoring, anomaly detection and exception generation
- reports — exports and management reports
- dashboard — operational KPIs and investigation views
- tests — validation, rules, RBAC, APIs and workflow tests

12. RBAC

- **Admin:** users, rules, thresholds, datasets and all analytics.
- **Auditor:** investigate exceptions, assign issues, add notes, update status and generate reports.
- **Analyst:** upload datasets, run analytics and view dashboards.
- **Viewer/Management:** read-only executive dashboards and reports.

13. Exception Investigation Workflow

Each detected issue becomes a structured Audit Exception with an ID, transaction, vendor, amount, violated rule, risk score, explanation, status, assigned auditor, review notes and resolution.

Recommended states: **OPEN** → **UNDER REVIEW** → **RESOLVED** or **OPEN** → **FALSE POSITIVE**.

Maintain an audit log for important actions, recording who changed what and when.

14. Dashboard Design

- **Executive Overview:** total transactions, transaction value, exceptions, high-risk exceptions, potential exposure, exception rate, monthly trends.
- **Financial Analysis:** transaction value, payment trends, vendor spend, department spend and top vendors.
- **Audit Exceptions:** exception type, severity, status, vendor, department, amount and risk score with filters.
- **Vendor Risk:** payment value, transaction count, average payment, exception count and risk.
- **Investigation:** transaction → invoice → purchase order → vendor → employee → approval → exception reason → risk score.

15. REST API Plan

- POST /api/upload/
- GET /api/transactions/
- GET /api/exceptions/
- GET /api/exceptions/{id}/
- GET /api/vendors/{id}/risk/
- GET /api/dashboard/kpis/
- GET /api/dashboard/trends/
- POST /api/audit/run/
- PATCH /api/exceptions/{id}/status/

Implement these with Django REST Framework only if you learn and use it; FastAPI is unnecessary for this project.

16. Testing, Validation and Security

- Unit-test ETL validation, audit rules, risk scoring, APIs, authentication, RBAC and exception workflow.
- Validate amounts, dates, IDs, required fields and uploaded files.
- Use Django authentication and permission checks.
- Use CSRF protection and safe ORM/parameterized database access.
- Restrict upload size/type and avoid exposing sensitive bank information.
- Mask sensitive values such as bank-account numbers in the UI.
- Log important errors and debugging information.

17. Step-by-Step Implementation Plan

Phase 1 — Domain understanding

Understand invoices, purchase orders, payments, vendors, expenses, controls, audit exceptions and risk.

Phase 2 — Synthetic data

Generate realistic datasets. Start around 1,000 records, validate the pipeline, then scale to 10,000+ if useful. Intentionally inject controlled exceptions and document them.

Phase 3 — PostgreSQL

Create normalized tables, foreign keys, constraints and useful indexes.

Phase 4 — Django foundation

Create project, settings, PostgreSQL connection, models, templates, static files and authentication.

Phase 5 — RBAC

Implement Admin, Auditor, Analyst and Viewer roles and test permissions.

Phase 6 — Upload

Build CSV/JSON/XLSX upload with file and schema validation.

Phase 7 — ETL

Implement extract(), transform(), validate(), load() using Pandas/NumPy and PostgreSQL.

Phase 8 — SQL analytics

Build and document 20–30 meaningful analytical queries.

Phase 9 — Audit rules

Implement duplicate, timing, approval, split-purchase, vendor, expense and relationship checks.

Phase 10 — Risk engine

Create a documented 0–100 risk-scoring system.

Phase 11 — Statistical analysis

Add distributions, percentiles, standard deviation and outlier analysis.

Phase 12 — Isolation Forest

Add unsupervised anomaly detection using transaction-level features.

Phase 13 — Result integration

Combine rule, statistical and ML signals into audit exceptions.

Phase 14 — APIs

Implement the required Django/DRF endpoints if using DRF.

Phase 15 — Dashboard

Build Django operational views and a Power BI analytical dashboard.

Phase 16 — Investigation workflow

Add assignment, notes, status changes, resolution and false-positive handling.

Phase 17 — Audit trail

Record user actions and important changes.

Phase 18 — Testing

Test ETL, rules, scoring, APIs, RBAC, validation and workflow.

Phase 19 — Documentation

Write README, architecture, schema, rules, API usage, screenshots, results, limitations and setup instructions.

18. Interview Questions You Should Be Ready For

- Why did you build this project?
- Why is an audit exception not the same as fraud?
- How did you identify duplicate invoices using SQL?
- Why did you use Django instead of another backend framework?
- How does the ETL pipeline validate and clean data?
- Why use Isolation Forest when you do not have labeled fraud data?
- How is the risk score calculated and explained?
- How do RBAC permissions work?

- How would you prevent a Viewer from modifying an exception?
- How did you test the audit rules?
- What would you do if the model produced too many false positives?
- How would you scale the application for millions of transactions?

19. Final Resume Positioning

Audit Analytics & Internal Control Monitoring | Python, Django, PostgreSQL, SQL, Pandas, Power BI

- Built an end-to-end audit analytics platform using Django, PostgreSQL, Python, and Pandas to ingest, validate, transform, and analyze financial transaction datasets.
- Developed SQL-based audit tests for duplicate invoices/payments, approval-threshold violations, split purchases, unusual transaction timing, and vendor activity, generating explainable risk-scored exceptions.
- Implemented statistical anomaly detection and Isolation Forest to identify unusual transaction patterns and prioritize exceptions for auditor review.
- Developed RBAC, REST APIs, exception workflows, audit trails, and Power BI dashboards for transaction investigation, risk analysis, and management-level KPI reporting.

Use these bullets only after the corresponding features are actually implemented and the metrics/results are measured.

20. Final Portfolio Strategy

This should be your flagship Data Analyst project. It reuses your current strengths—Python, SQL, Django, PostgreSQL, Pandas, Scikit-learn, ETL, RBAC, testing, validation and documentation—while adding Power BI and, if implemented, Django REST Framework. Do not add FastAPI or unrelated technologies merely to make the stack look larger.

The strongest story is: **business problem** → **data ingestion** → **data quality** → **SQL analysis** → **audit controls** → **anomaly detection** → **explainable risk** → **investigation** → **dashboard** → **report.**